simple_rl — Detailed Module Design

Online Reinforcement Learning for Terminal Agents on Apple Silicon

OpenClaw-RL Project

2026-03-27

Contents

1 System Overview

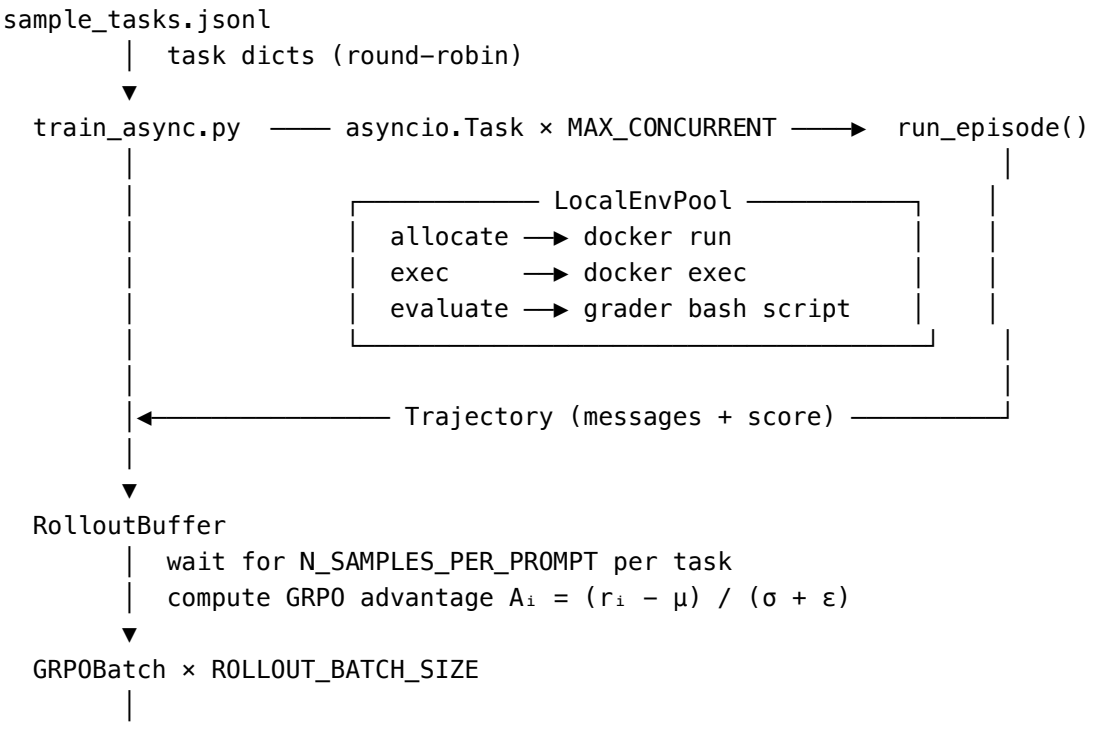
`simple_rl` is a self-contained, **Apple Silicon–native** online reinforcement learning framework for training terminal agents. It deliberately replaces the distributed infrastructure from earlier iterations (Ray, CAMEL, Router Server, remote Pool Workers) with a single-process AsyncIO design that runs entirely on one Mac.

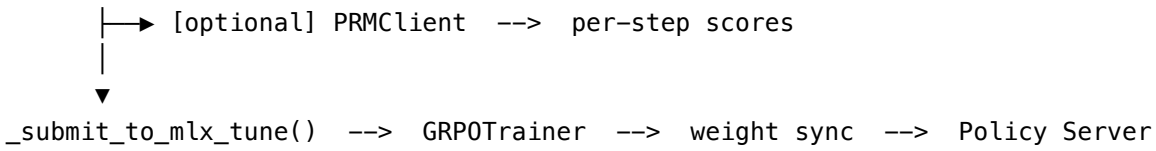
1.1 Design Philosophy

Principle	Old stack	simple_rl
Concurrency	Ray distributed tasks	`asyncio.Task` per episode
Inter-process comms	HTTP between Pool/Router/Worker servers	In-process function calls
Container management	Remote pool server over HTTP	`LocalEnvPool` — in-process subprocess
Model serving	External vLLM or TGI	oMLX / `mlx_lm.server` — same Mac
Weight updates	Serialise –> upload –> reload	In-place MLX array update
Dependencies	Ray, CAMEL, Pydantic servers	`openai`, `asyncio`, `subprocess`

The result is a system that is easier to debug (single log stream, one Python process, `asyncio` traces), cheaper to run (no distributed infrastructure), and purpose-built for the unified memory model of Apple Silicon.

1.2 High-Level Data Flow




├──► [optional] PRMClient --> per-step scores
▼
_submit_to_mlx_tune() --> GRPOTrainer --> weight sync --> Policy Server

2 Module Reference

2.1 `config.py` — Centralised Configuration

File: `simple_rl/config.py` **Role:** Single source of truth for every tunable parameter. All values read from environment variables with safe defaults.

2.1.1 Parameters

Variable	Default	Environment Variable	Description
`POLICY_URL`	`http://localhost:8000/v1`	`POLICY_URL`	OpenAI-compat base URL for policy model
`POLICY_MODEL`	`Qwen3-8B-4bit`	`POLICY_MODEL`	Model name sent in API requests
`PRM_URL`	`http://localhost:8001/v1`	`PRM_URL`	Base URL for optional PRM slot
`PRM_MODEL`	`Qwen3-4B-4bit`	`PRM_MODEL`	PRM model name
`PRM_ENABLE`	`False`	`PRM_ENABLE=1`	Enable per-step PRM scoring
`PRM_M`	`3`	`PRM_M`	Majority-vote count per step
`DOCKER_IMAGE`	`ubuntu:22.04`	`DOCKER_IMAGE`	Base image for episode containers
`MAX_CONCURRENT`	`4`	`MAX_CONCURRENT`	Max parallel episodes
`IDLE_TIMEOUT`	`600` s	`IDLE_TIMEOUT`	Seconds before idle container eviction
`EXEC_TIMEOUT`	`30.0` s	`EXEC_TIMEOUT`	Per-command execution timeout
`EVAL_TIMEOUT`	`60.0` s	`EVAL_TIMEOUT`	Grader script timeout
`MAX_TURNS`	`20`	`MAX_TURNS`	Max agent turns per episode
`CONTEXT_LEN`	`16384` chars	`CONTEXT_LEN`	Character budget for message history
`N_SAMPLES_PER_PROMPT`	`8`	`N_SAMPLES_PER_PROMPT`	Trajectories to collect per task before GRPO
`ROLLOUT_BATCH_SIZE`	`4`	`ROLLOUT_BATCH_SIZE`	GRPO groups per training round
`KL_LOSS_COEF`	`0.01`	`KL_LOSS_COEF`	KL penalty coefficient for GRPOTrainer
`LORA_RANK`	`16`	`LORA_RANK`	LoRA adapter rank
`DATASET_PATH`	`data/sample_tasks.jsonl`	`DATASET_PATH`	Path to task dataset
`LOG_DIR`	`logs`	`LOG_DIR`	Directory for trajectory logs and GRPO batches
`WANDB_PROJECT`	`""` (disabled)	`WANDB_PROJECT`	W&B project name (empty = no W&B)

2.1.2 Design Notes

- Every module imports `from . import config` and reads values at call-time, so changing an env var before importing has the expected effect.
- No config file is required — the defaults are tuned for a 16 GB M2 Mac running Qwen3-8B-4bit.

2.2 `terminal_env.py` — Docker Container Wrapper

File: `simple_rl/terminal_env.py` **Role:** Thin async wrapper around a single Docker container. One `TerminalEnv` instance = one RL episode's execution environment.

2.2.1 Class: `TerminalEnv`

```
TerminalEnv
├── docker_image   : str   (default: config.DOCKER_IMAGE)
├── container_name : str   (auto: "simple-rl-<10 hex chars>")
└── _running       : bool  (internal state flag)
```

2.2.2 Lifecycle

`start(task)` ⟶ `exec(bash_cmd)` ⟶ `evaluate(grader)` ⟶ `close()`

Each method is an `async def` coroutine built on `asyncio.create_subprocess_exec`.

2.2.2.1 `start(task: dict) --> None` Runs:

```
docker run --rm -d --name <container_name> \
           --network none \
           ubuntu:22.04 sleep infinity
```

Key flags:

- `--rm`: container is auto-deleted when stopped (no dangling containers)
- `-d`: detached mode — Docker returns immediately with the container ID
- `--network none`: episode is fully sandboxed — no outbound internet access
- `sleep infinity`: keeps the container alive until explicitly removed

2.2.2.2 `exec(bash_cmd: str, timeout: float) --> str` Runs the agent's bash command via:

```
docker exec <container_name> bash -c "<bash_cmd>"
```

Behaviour:

- Merges `stdout` and `stderr` into one string (matches a real terminal)
- Caps output at `_MAX_OUTPUT_BYTES = 4096` — takes the **tail** to preserve the most recent output when commands produce large output
- On `asyncio.TimeoutError`: sends `pkill -9 bash` inside the container (best-effort cleanup), returns `"[TIMEOUT] Command exceeded Xs limit."`
- On any other exception: returns `"[EXEC_ERROR] ExcType: message"`
- Never raises — the agent loop always gets a string back

2.2.2.3 `evaluate(grader_script: str, timeout: float) --> float` Runs the task's grader bash script inside the container (same `exec()` path), then parses the **last non-empty line** of output as a `float`.

- Clamps result to `[0.0, 1.0]`
- Returns `0.0` if no parseable float is found (with a warning log)
- Grader scripts are co-located with task definitions in `sample_tasks.jsonl`

2.2.2.4 `close() --> None` Runs `docker rm -f <container_name>` with a 15-second timeout. Sets `_running = False` before the subprocess call so repeated `close()` calls are idempotent.

2.2.3 Output Capping — Why the Tail?

When a command produces more than 4 096 bytes, keeping the tail (most recent output) is almost always more useful than the head — shell commands typically emit diagnostic output and results at the end, and the agent's next decision should be based on what the command most recently produced.

2.3 `local_env_pool.py` — In-Process Container Pool

File: `simple_rl/local_env_pool.py` **Role:** Manages a bounded pool of `TerminalEnv` Docker containers, replacing the HTTP-based Router Server + Pool Server pair from the old architecture. Everything runs in-process — no network hop between the training loop and the container pool.

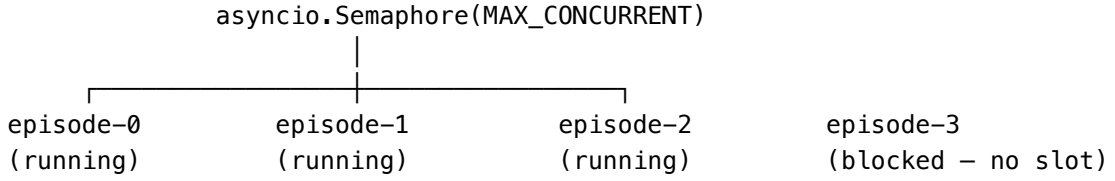
2.3.1 Class: `LocalEnvPool`

```
LocalEnvPool
├── _max          : int                        (max concurrent episodes)
├── _docker_image : str
├── _idle_timeout : float
├── _sem          : asyncio.Semaphore(_max)    (capacity gate)
├── _slots        : Dict[lease_id --> _Slot]     (active containers)
├── _lock         : asyncio.Lock               (protects _slots)
└── _reaper       : asyncio.Task               (idle container eviction)
```

2.3.2 `_Slot` Dataclass

```
@dataclass
class _Slot:
    env       : TerminalEnv
    task      : dict
    lease_id  : str
    last_used : float  # monotonic timestamp, updated on every exec()
```

2.3.3 Concurrency Model



`allocate()` calls `await self._sem.acquire()`. If all `MAX_CONCURRENT` slots are in use, the caller blocks in the event loop (without spinning) until a slot is released by `close()`.

2.3.4 Public API

Method	Returns	Description
`start()`	`None`	Starts the idle-reaper background task
`stop()`	`None`	Cancels reaper, closes all containers
`allocate(task)`	`lease_id: str`	Acquires slot, starts container, returns opaque lease
`exec(lease_id, cmd)`	`str`	Delegates to `TerminalEnv.exec()`
`evaluate(lease_id)`	`float`	Reads grader from task dict, delegates to `TerminalEnv.evaluate()`
`close(lease_id)`	`None`	Stops container, releases semaphore

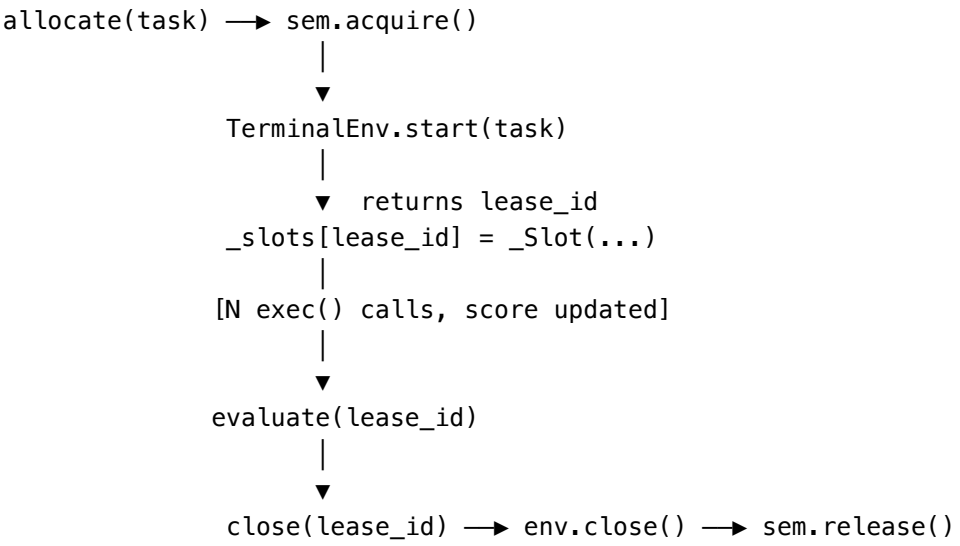
2.3.5 Idle Reaper

`_idle_reaper()` runs as a background `asyncio.Task` sleeping 60 seconds between passes. On each wake it computes:

```
stale = [lid for lid, slot in self._slots.items()
         if (now - slot.last_used) > IDLE_TIMEOUT]
```

Any slot that has not seen an `exec()` call within `IDLE_TIMEOUT` seconds (default 600 s / 10 min) is force-closed. This prevents orphaned containers when an episode stalls or is cancelled abnormally.

2.3.6 Lease Lifecycle



2.4 `agent_loop.py` — Multi-Turn Agent Loop

File: `simple_rl/agent_loop.py` **Role:** Implements the per-episode agent coroutine. Manages the conversation with the policy model, extracts bash commands, feeds observations back, and detects task completion.

2.4.1 Tool Protocol

The system prompt instructs the model to use exactly two primitives:

```
<bash>
command here
</bash>
```

to execute a shell command, and the literal string

```
TASK_COMPLETE
```

to signal that the task is done. Observations are injected as user messages:

```
<observation>
stdout + stderr here
</observation>
```

This XML-tag convention is simple, unambiguous, and parseable with a single regex — no JSON schema or function-calling API required.

2.4.2 `Trajectory` Dataclass

```
@dataclass
class Trajectory:
    task              : dict            # original task definition
    messages          : List[dict]      # full conversation history
    turn_scores       : List[float]     # per-step PRM scores (optional)
    score             : float           # final grader score [0, 1]
    n_turns           : int             # number of agent turns taken
    prompt_tokens     : int             # cumulative prompt tokens used
    completion_tokens : int             # cumulative completion tokens
    elapsed_s         : float           # wall-clock episode time
    done              : bool            # True if TASK_COMPLETE was emitted
    error             : Optional[str]   # error message if episode failed
```

This dataclass is the primary unit passed between the agent loop, rollout buffer, PRM client, and training pipeline.

2.4.3 `_parse_bash_cmd(text)` — Command Extraction

Accepts two formats to be robust to model behaviour:

1. **XML tags** (preferred): `<bash>…</bash>`
2. **Markdown fences** (fallback): ```` ```bash\n…\n``` ```` or ```` ```sh\n…\n``` ````

If neither format is found, the agent injects an `[AGENT_HINT]` observation telling the model how to format its command rather than silently failing.

2.4.4 `_trim_messages(messages, budget)` — Context Window Management

Keeps the total character length of all message content within `CONTEXT_LEN` (default 16 384 chars) by dropping the **oldest assistant+observation pairs** from the middle of the message list.

```
messages = [
    [0] system prompt          ← always kept
    [1] initial user task      ← always kept
    [2] assistant turn 1       ← dropped first when over budget
    [3] observation 1          ← dropped with its pair
    [4] assistant turn 2       ← kept until budget forces drop
    [5] observation 2
    ...
]
```

Pairs are always dropped together to maintain a valid alternating role structure (`system --> user --> assistant --> user --> …`).

2.4.5 `run_episode(task, env_pool, ...) --> Trajectory`

The main coroutine executing one complete RL episode:

```
allocate(task)
    |
    ▼
for turn in range(max_turns):
    ├── _trim_messages()
    ├── POST /v1/chat/completions  -->  assistant text
    ├── accumulate token usage
    ├── if TASK_COMPLETE in text  -->  break (done=True)
    ├── _parse_bash_cmd(text)
    |       ├── found  -->  env_pool.exec(lease, cmd)  -->  observation
    |       └── not found  -->  inject [AGENT_HINT]
    └── append observation as user message
    |
evaluate(lease)  -->  score
close(lease)     -->  sem.release()
return Trajectory
```

All exceptions are caught in a `try/finally` block so `close(lease)` is **always** called — preventing semaphore leaks that would deadlock the pool.

2.4.6 Policy API

The agent uses `openai.AsyncOpenAI` pointed at `POLICY_URL`. The request is:

```
await client.chat.completions.create(
    model    = policy_model,
```

```
    messages = trimmed,
    max_tokens   = 1024,
    temperature  = 0.7,
)
```

Any `openai.OpenAIError` terminates the episode with `traj.error` set — the episode still returns a `Trajectory` with `score=0.0` so the rollout buffer can handle it gracefully.

2.5 `rollout_buffer.py` — GRPO Rollout Buffer

File: `simple_rl/rollout_buffer.py` **Role:** Accumulates trajectories from concurrent episodes, groups them by task, computes GRPO advantages, and queues completed batches for the trainer.

2.5.1 GRPO Advantage Computation

Group Relative Policy Optimisation (GRPO) scores trajectories relative to other attempts at the same task:

$$A_i = \frac{r_i - \bar{r}}{\hat{\sigma}(r) + \varepsilon}$$

Where:

- r_i = final grader score for trajectory i
- $\bar{r}$ = mean score across the group of `N_SAMPLES_PER_PROMPT` trajectories
- $\hat{\sigma}(r)$ = sample standard deviation across the group
- $\varepsilon = 10^{-8}$ = numerical stability constant

Why relative scoring? A fixed reward threshold (e.g. "1.0 = good") would provide no gradient signal when the model always scores above or below it. Normalising within the group amplifies the signal from the best-vs-worst trajectories regardless of the absolute score level.

Edge case — identical scores: When all trajectories in a group achieve the same score, `stdev = 0`. The formula produces `A_i = 0` for all samples (via the ε guard), contributing zero gradient — which is correct: there is nothing to differentiate in the group.

2.5.2 Data Types

```
GRPOSample
├── trajectory : Trajectory
└── advantage  : float        # Aᵢ for this sample

GRPOBatch
├── task_id    : str
├── samples    : List[GRPOSample]
├── mean_score : float  (property — average score across group)
└── mean_advantage : float  (property — should be ≈ 0 by construction)
```

2.5.3 `RolloutBuffer` Internals

```
_pending : Dict[task_id --> List[Trajectory]]   # accumulator per task
_lock    : asyncio.Lock                         # protects _pending
_queue   : asyncio.Queue[GRPOBatch]            # completed batches
```

2.5.3.1 `add(task_id, traj, prm_client) --> Optional[GRPOBatch]`

1. Acquire `_lock`; append `traj` to `_pending[task_id]`
2. If `len(group) < N_SAMPLES_PER_PROMPT` –> return `None` (still accumulating)

3. Pop the complete group from `_pending` (release lock)
4. If `prm_client` is provided, concurrently score all trajectories via `asyncio.gather()` — this is done **outside the lock** because PRM scoring involves I/O and may be slow
5. Call `_make_batch()` –> compute advantages –> return `GRPOBatch`

2.5.3.2 `get_training_batches(batch_size) --> List[GRPOBatch]` Blocks on `asyncio.Queue.get()` until `batch_size` complete groups are available. Used by the training loop to wait for a full round of data.

2.5.4 Isolation Guarantee

Each task's trajectories are accumulated independently in `_pending`. Tasks never contaminate each other's advantage groups — a batch from `task_A` will only contain trajectories that all attempted `task_A`'s prompt.

2.6 `prm_client.py` — Process Reward Model Client

File: `simple_rl/prm_client.py` **Role:** Optional module that assigns a quality score to each individual `(command, observation)` step within a trajectory using a second, smaller LLM served on a separate port.

Enable with: `PRM_ENABLE=1`

2.6.1 Why a PRM?

The terminal grader only produces one signal at the **end** of the episode (pass/fail or partial credit). Many turns may have contributed positively or negatively to that outcome — the grader cannot distinguish them. A PRM can assign **dense per-step rewards** that provide much richer gradient signal, particularly for long episodes where the final score poorly attributes credit to early actions.

2.6.2 Architecture

```
PRMClient
├── _model  : str              (config.PRM_MODEL, default Qwen3-4B-4bit)
├── _m      : int              (config.PRM_M, default 3 — votes per step)
└── _client : openai.AsyncOpenAI (base_url=config.PRM_URL = localhost:8081)
```

The PRM is served as a second `mlx_lm.server` instance on port 8081 — same OpenAI-compatible API as the policy server, just a smaller model.

2.6.3 `score_step(task_desc, command, output) --> float`

Formats a zero-shot prompt:

```
You are a step evaluator for a terminal agent.
Task: {task_desc[:400]}
Bash command: {command[:300]}
Output: {output[:600]}
Reply with a single float between 0.0 and 1.0.
```

Calls the PRM model `_m` times with `temperature=0.3`, parses the first float token from each response, and returns the mean. If all calls fail, returns `0.5` (neutral).

Majority vote over `_m` samples reduces per-call noise from a small model. `temperature=0.3` keeps responses focused while allowing some variation.

2.6.4 `score_trajectory(traj) --> List[float]`

Iterates over `traj.messages`, finding every `assistant` message and pairing it with the following `user` message (the observation). Calls `score_step()` on each pair sequentially (to avoid hammering the PRM server).

The returned list has one float per agent turn, in chronological order. These scores are stored in `traj.turn_scores` by the rollout buffer.

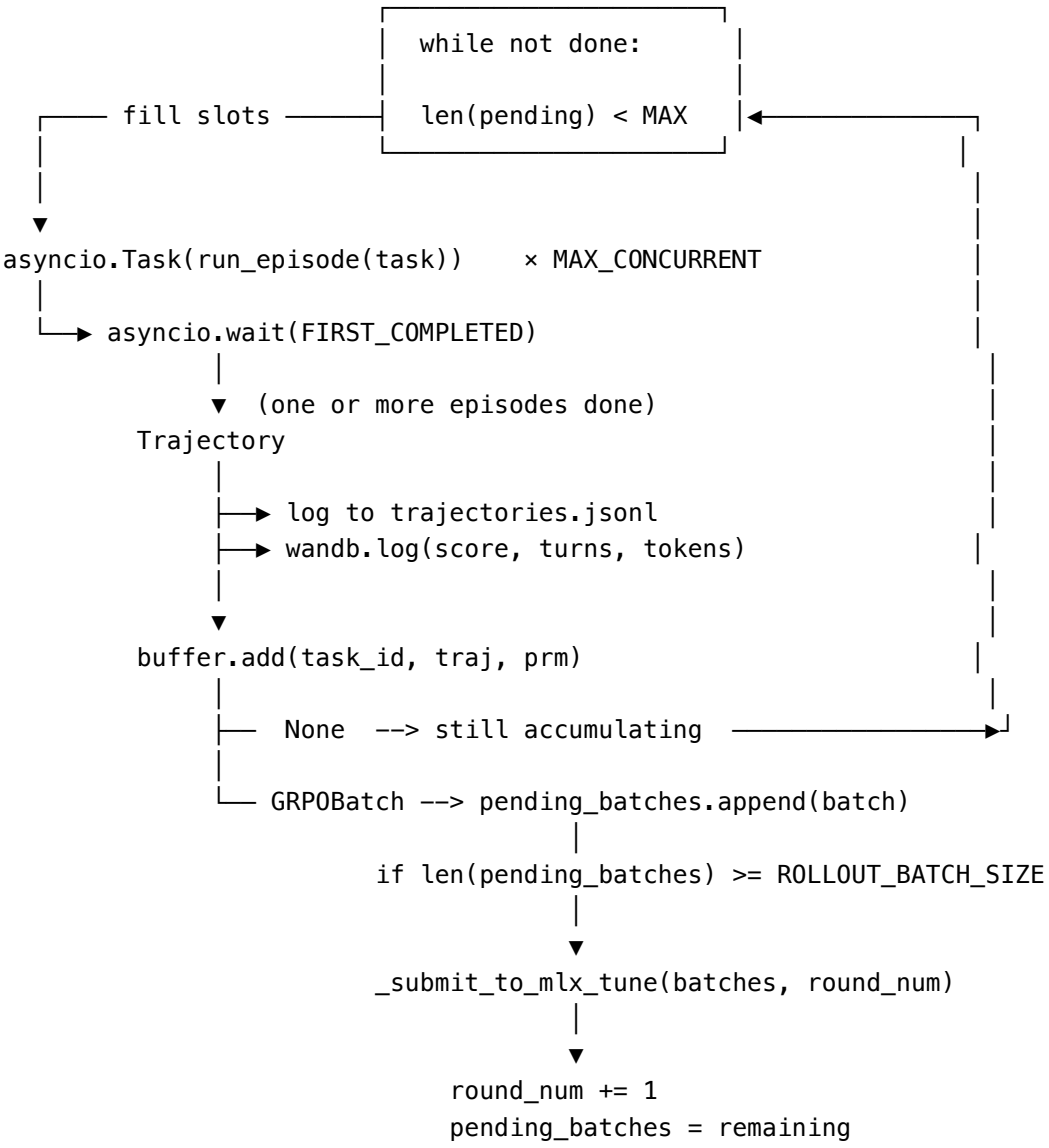
2.6.5 Integration with GRPO

Per-step scores are stored on the `Trajectory` but the **GRPO advantage is still computed from the final episode score** (`traj.score`), not from `turn_scores`. This is intentional: PRM scores enrich the training signal available to the trainer (e.g. for credit assignment in policy gradient variants) but do not alter the group normalisation used in GRPO.

2.7 `train_async.py` — Training Orchestrator

File: `simple_rl/train_async.py` **Role:** Top-level `asyncio` event loop that drives the full online RL training cycle. Dispatches episodes, feeds the rollout buffer, and fires training rounds when enough data is ready.

2.7.1 Training Loop State Machine



2.7.2 `asyncio.wait(FIRST_COMPLETED)`

Rather than `asyncio.gather()` (which waits for all), the loop uses `asyncio.wait(..., return_when=FIRST_COMPLETED)`. This means:

- Episodes complete at different rates without blocking each other
- The slowest episode never holds up reward processing for faster ones
- The concurrency slot is recycled immediately when any episode finishes

After `wait()` returns, the loop immediately refills the slot with a new episode — keeping all `MAX_CONCURRENT` slots busy at all times.

2.7.3 `load_dataset(path) --> List[dict]`

Reads a JSONL file, skipping blank lines and malformed JSON (with a warning). Raises `ValueError` if no tasks are loaded. The training loop cycles through tasks round-robin: `task = tasks[task_idx % len(tasks)]`.

2.7.4 `_submit_to_mlx_tune(batches, round_num, log_dir)` — Training Stub

This function is the **integration point** for the actual MLX gradient update. In its current form it:

1. Logs summary statistics (`mean_score`, `mean_advantage`, `n_samples`)
2. Writes each sample to `logs/grpo_batch_r<NNNN>.jsonl` with fields: `task_id`, `advantage`, `score`, `n_turns`, `messages`, `turn_scores`

To wire to real `mlx-tune`:

```
# 1. The batch is already serialised to grpo_batch_rNNNN.jsonl
# 2. Call mlx_lm's GRPOTrainer Python API:
from mlx_lm.tuner.grpo_trainer import GRPOTrainer
trainer = GRPOTrainer(model, tokenizer, args)
trainer.train_step(batch)
# 3. The trainer updates weights in-place via MLX array mutation
# 4. The shared memory model reference in the policy server reflects
#    the update immediately — no serialise/reload cycle needed
```

2.7.5 Logging and Observability

Trajectory log (`logs/trajectories.jsonl`) — one JSON record per episode:

```
{"task_name": "hello_world", "score": 1.0, "n_turns": 2,
 "done": true, "prompt_tokens": 412, "completion_tokens": 87,
 "elapsed_s": 3.14, "error": null, "step": 1}
```

GRPO batch logs (`logs/grpo_batch_r0001.jsonl`) — one record per sample in each training round, including the full message history and advantages.

Weights & Biases — if `WANDB_PROJECT` is set and `wandb` is installed, per-step metrics (`score`, `turns`, `tokens`) and per-round metrics (`round_mean_score`) are logged.

2.7.6 Graceful Shutdown

On `KeyboardInterrupt` (or `max_rounds` reached), the `finally` block:

1. Cancels all in-flight `asyncio.Task` episode coroutines
2. `await asyncio.gather(*pending_tasks, return_exceptions=True)` — drains
3. `await env_pool.stop()` — cancels reaper, removes all Docker containers
4. Closes the trajectory log file
5. Calls `wandb.finish()` if W&B is active

2.8 `data/sample_tasks.jsonl` — Task Dataset

File: `simple_rl/data/sample_tasks.jsonl` **Role:** Provides eight self-contained bash tasks for development, testing, and dry-run validation.

2.8.1 Task Schema

Each line is a JSON object:

```
{
  "task_name":    "unique_identifier",
  "instruction":  "Natural language description of the task",
  "grader":       "bash script that prints a float 0.0–1.0",
  "docker_image": "ubuntu:22.04"
}
```

2.8.2 Included Tasks

Task Name	Instruction Summary	Grader Check
`hello_world`	Print "Hello, World!"	`grep -q 'Hello, World!' /tmp/hello.txt`
`count_files`	Count files in `/etc`	Parse integer from `/tmp/count.txt`, score by distance from actual count
`find_large_files`	Find files >1 MB in `/usr`	Check at least one result in `/tmp/large.txt`
`word_count`	Count words in `/etc/os-release`	Compare against `wc -w` baseline
`create_directory_tree`	Create nested `a/b/c/d/e` under `/tmp`	`test -d /tmp/a/b/c/d/e`
`reverse_string`	Reverse "Hello, World!"	Exact string match in `/tmp/reversed.txt`
`sort_numbers`	Sort `[3,1,4,1,5,9]`	Exact match against expected sorted list
`fibonacci`	Compute first 10 Fibonacci numbers	Exact match against `[0,1,1,2,3,5,8,13,21,34]`

2.8.3 Grader Design Pattern

Graders follow a strict convention:

- They run **inside the same container** as the episode (no external calls)
- They check for expected side effects (files created, stdout written)
- They print **exactly one float** on the last non-empty line
- Full credit (1.0) for exact success; partial credit (0.5) for partial work; 0.0 for failure or missing output

2.8.4 Adding Custom Tasks

```
{
```

```
  "task_name": "install_package",
  "instruction": "Install the 'jq' package using apt-get and verify it works by running 'jq --version'.",
  "grader": "which jq >/dev/null 2>&1 && jq --version >/dev/null 2>&1 && echo 1.0 || echo 0.0",
  "docker_image": "ubuntu:22.04"
}
```

Any bash one-liner or multi-line heredoc that ends with `echo <float>` is valid as a grader.

3 Cross-Cutting Concerns

3.1 Concurrency Architecture

```
Python Process (single OS thread)
│
└── asyncio Event Loop
      │
      ├── asyncio.Task: episode-0   (run_episode coroutine)
      ├── asyncio.Task: episode-1
      ├── asyncio.Task: episode-2
      ├── asyncio.Task: episode-3
      ├── asyncio.Task: env-pool-reaper
      │
      └── I/O multiplexing:
            ├── asyncio.create_subprocess_exec  (docker run / exec / rm)
            ├── openai.AsyncOpenAI HTTP client  (policy / PRM API calls)
            └── asyncio.Queue / asyncio.Semaphore
```

Because the event loop is single-threaded, there are **no race conditions** on shared Python objects — only I/O operations yield control. The `asyncio.Lock` on `LocalEnvPool._slots` and `RolloutBuffer._pending` guards the small critical sections where multiple coroutines update shared dicts.

3.2 Error Propagation Strategy

Component	On Error	Behaviour
`TerminalEnv.exec`	Timeout, process error	Returns string `[TIMEOUT]`/`[EXEC_ERROR]` — never raises
`TerminalEnv.close`	Docker error	Logs warning, continues (best-effort cleanup)
`run_episode`	Any unhandled exception	Sets `traj.error`, returns Trajectory with `score=0.0`
`run_episode`	Policy API error	Sets `traj.error`, breaks the turn loop
`LocalEnvPool.allocate`	Docker start failure	Releases semaphore, re-raises — episode fails gracefully
`RolloutBuffer._score_with_prm`	PRM error	Logs warning, `turn_scores` stays empty — never fails the batch
`train` loop	`KeyboardInterrupt`	Graceful shutdown: cancel tasks, stop pool, close logs

3.3 Memory Footprint

On a 16 GB Apple Silicon Mac with the default configuration:

Component	Memory
Qwen3-8B-4bit policy model	≈ 5 GB
Qwen3-4B-4bit PRM model (optional)	≈ 3 GB
4 Docker containers (ubuntu:22.04)	≈ 50–100 MB
Python process + asyncio overhead	≈ 100–200 MB
Total (with PRM)	≈ **8.5 GB**
Total (without PRM)	≈ **5.5 GB**

The 4-bit quantised models use MLX's unified memory — they share the same physical DRAM as the CPU and GPU, so there is no PCIe transfer overhead.

3.4 Testing Strategy

Tests are split into two categories by `pytest` markers:

Unit tests (`@pytest.mark.unit`) — no Docker, no live model server:

- Mock `asyncio.create_subprocess_exec` to return fake stdout/stderr
- Test all code paths including timeouts, output capping, error returns
- Run in CI with `pytest -m unit` — fast (<5 s total)

Integration tests (`@pytest.mark.integration`) — require Docker daemon:

- Start real containers, execute real commands, verify real output
- Run locally with `pytest -m integration`

The default `pytest.ini` configuration runs only unit tests: `addopts = -v -m "not integration"`.

4 Deployment Reference

4.1 Minimal Startup

```
# 1. Start the policy server (oMLX)
mlx_lm.server --model mlx-community/Qwen3-8B-4bit --port 8080

# 2. (Optional) Start the PRM server
mlx_lm.server --model mlx-community/Qwen3-4B-4bit --port 8081

# 3. Run training
cd /path/to/OpenClaw-RL
POLICY_URL=http://localhost:8080/v1 bash simple_rl/run.sh

# Or dry-run (2 rounds, no real LLM needed for unit-test validation)
POLICY_URL=http://localhost:8080/v1 bash simple_rl/run.sh --dry-run
```

4.2 Environment Variable Quick Reference

```
# Core
export POLICY_URL=http://localhost:8080/v1
export POLICY_MODEL=Qwen3-8B-4bit
export MAX_CONCURRENT=4          # parallel episodes
export N_SAMPLES_PER_PROMPT=8    # trajectories per task before GRPO
export ROLLOUT_BATCH_SIZE=4      # task groups per training round
export MAX_TURNS=20              # max agent turns per episode

# Tuning
export CONTEXT_LEN=16384         # char budget for message history
export EXEC_TIMEOUT=30           # seconds per bash command
export EVAL_TIMEOUT=60           # seconds for grader script
export IDLE_TIMEOUT=600          # seconds before container eviction

# PRM (optional)
export PRM_ENABLE=1
export PRM_URL=http://localhost:8081/v1
export PRM_MODEL=Qwen3-4B-4bit
export PRM_M=3                   # votes per step

# Logging
export LOG_DIR=logs
export WANDB_PROJECT=terminal-rl
```

4.3 File Layout

```
simple_rl/
├── __init__.py
├── config.py           ← all configuration, env-var overridable
├── terminal_env.py     ← Docker container wrapper
```

```
├── local_env_pool.py   ← bounded async container pool
├── agent_loop.py       ← multi-turn agent + Trajectory dataclass
├── rollout_buffer.py   ← GRPO buffer + GRPOBatch
├── prm_client.py       ← optional per-step scoring
├── train_async.py      ← main training loop + CLI
├── run.sh              ← startup script with pre-flight checks
├── pytest.ini
├── data/
│   └── sample_tasks.jsonl
├── tests/
│   ├── conftest.py
│   ├── test_terminal_env.py
│   ├── test_local_env_pool.py
│   ├── test_agent_loop.py
│   └── test_rollout_buffer.py
└── assets/
    ├── simple-rl-architecture.mmd  ← detailed Mermaid source
    ├── simple-rl-architecture.png  ← rendered architecture diagram
    ├── simple-rl-overview.mmd      ← simplified overview source
    └── simple-rl-overview.png      ← rendered overview diagram
```

Document generated from source: `simple_rl/` — OpenClaw-RL project.